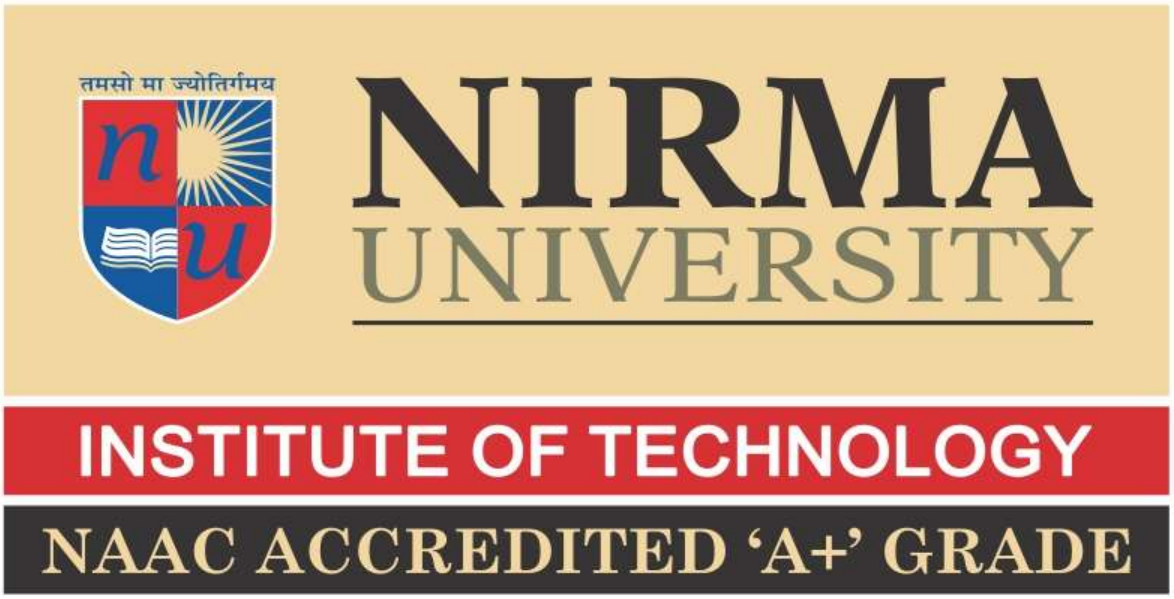
RISC-V Processor with Custom AES Instruction Set

3EC503CC24-Computer Architecture

Special Assignment Report



INSTITUTE OF TECHNOLOGY , NIRMA UNIVERSITY

Faculty : Dr. Dhaval Shah

Submitted By : Jiyan Patel (23BEC137) and Mihir Patel (23BEC141)

Abstract :

With the rapid growth of Internet of Things (IoT) devices and embedded systems, secure communication has become an essential requirement. Cryptographic algorithms such as Advanced Encryption Standard (AES) are widely used to ensure confidentiality and data integrity. However, implementing AES purely in software on general-purpose processors results in higher latency and increased computational overhead. This project presents the design and implementation of a single-cycle RISC-V processor integrated with a hardware AES cryptographic accelerator. The processor is implemented in Verilog HDL and supports both standard RISC-V instructions and custom AES instructions. Dedicated hardware modules are designed for AES operations such as SubBytes, ShiftRows, MixColumns, and AddRoundKey. The integration of AES functionality inside the processor significantly improves encryption performance and reduces execution cycles compared to software implementations. Simulation results demonstrate the correctness of the processor operation and successful encryption using AES. The proposed architecture provides a secure and efficient solution for embedded and IoT applications requiring hardware-accelerated cryptography.

Keywords :

AES Encryption, Cryptographic Processor, Embedded Security, Hardware Acceleration, RISC-V, Verilog HDL

1. Introduction:

In modern digital systems, large amounts of sensitive information such as financial data, personal information, and confidential communications are transmitted and stored electronically. Without proper protection, this data is vulnerable to unauthorized access, modification, and cyber attacks. Cryptography provides techniques for securing information by converting plaintext into encrypted data that can only be accessed by authorized users. As digital communication and internet-based applications continue to expand, cryptography has become a fundamental component in securing modern computing systems and embedded devices [5].

The **Advanced Encryption Standard (AES)** is a widely used symmetric key encryption algorithm designed to protect electronic data. Due to its strong security and efficient implementation in both hardware and software, AES is widely used in secure communication systems, financial applications, wireless networks, and data protection systems.

RISC-V is an open-source instruction set architecture (ISA) designed to provide a flexible and extensible platform for processor development. Unlike proprietary architectures, RISC-V allows researchers and developers to design customized processors and extend the instruction set to support specialized applications. The architecture follows the Reduced Instruction Set Computing (RISC) design philosophy, which simplifies instructions to achieve efficient hardware implementation and improved performance [4].

The main objective of this project is to design and implement a single-cycle RISC-V processor integrated with an AES cryptographic module using hardware description language (Verilog). The processor performs standard RISC-V instruction execution while supporting AES-based encryption functionality for secure data processing.

2. Literature Survey :

2.1 Advanced Encryption Standard (AES)

The Advanced Encryption Standard (AES) is a symmetric key encryption algorithm widely used for securing electronic data. It was standardized by the National Institute of Standards and Technology (NIST) in 2001 as FIPS Publication 197. AES is based on the Rijndael cipher developed by Joan Daemen and Vincent Rijmen. The algorithm operates on a fixed block size of 128 bits and supports key sizes of 128, 192, and 256 bits. AES performs encryption through several transformation stages including SubBytes, ShiftRows, MixColumns, and AddRoundKey [1]. These transformations provide strong security against various cryptographic attacks while maintaining efficient hardware and software implementation. Due to its robustness and performance, AES is widely used in applications such as wireless networks, secure communications, and data protection systems .

2.2 Hardware Implementation of AES

Hardware implementations of AES have been widely studied to improve encryption speed and reduce computational overhead compared to software implementations. FPGA-based AES implementations provide high performance and flexibility for cryptographic systems. Many research works have focused on optimizing AES architecture to achieve faster encryption throughput while minimizing hardware resources and power consumption. Hardware-based AES modules typically implement core operations such as SubBytes using S-box lookup tables, ShiftRows through byte permutation, and MixColumns using Galois field arithmetic. These hardware accelerators significantly enhance the performance of encryption systems in embedded devices and high-speed communication applications [6].

2.3 RISC-V Processor Architecture

RISC-V is an open-source instruction set architecture designed to provide flexibility and scalability for processor design. It follows the Reduced Instruction Set Computing (RISC) principle, which simplifies instruction formats and improves hardware efficiency. Unlike proprietary architectures, RISC-V allows researchers and developers to extend the instruction set with custom instructions tailored for specific applications. The base integer instruction set includes standard formats such as R-type, I-type, S-type, B-type, U-type, and J-type instructions, which enable efficient implementation of arithmetic, logical, and memory operations [3].

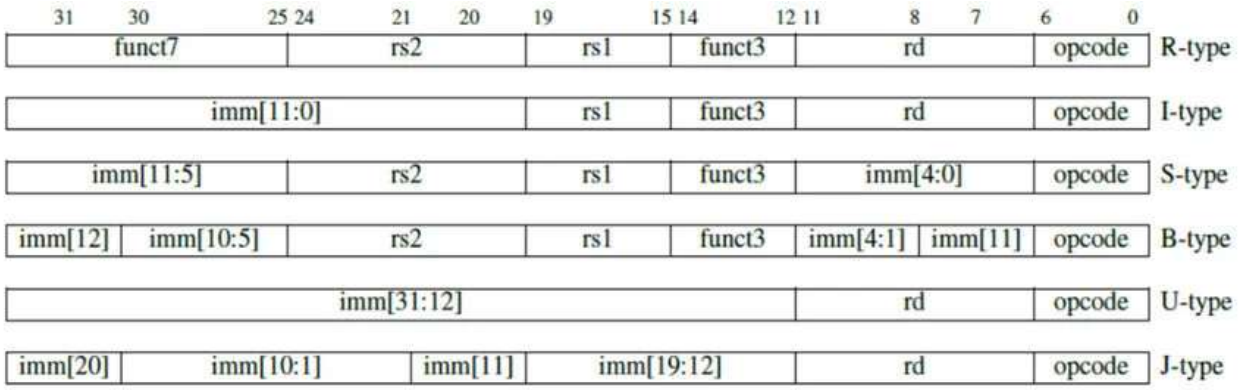


Figure 1 : RV32I instruction format[10]

2.4 Integration of Cryptography with RISC-V Processors

Recent research has explored the integration of cryptographic algorithms within processor architectures to enhance security and performance. Implementing AES as a hardware accelerator inside a RISC-V processor allows encryption operations to be executed more efficiently than traditional software-based implementations. Custom instruction extensions can be added to the RISC-V instruction set to support cryptographic operations, enabling faster execution of encryption rounds. Such processor architectures are particularly useful for embedded systems, IoT devices, and secure communication systems where both performance and security are critical requirements [5].

2.5 FPGA-Based Secure Processor Implementations

Field Programmable Gate Arrays (FPGAs) are widely used for prototyping and implementing processor architectures with cryptographic functionality. FPGA platforms allow designers to test and verify processor designs before hardware deployment. Tools such as **Xilinx Vivado Design Suite** enable synthesis, simulation, and verification of hardware modules developed using hardware description languages like Verilog or VHDL [8]. FPGA-based implementations of RISC-V processors integrated with AES modules have demonstrated improved encryption performance and hardware efficiency, making them suitable for secure embedded systems and high-performance computing applications.

3. Limitations of Existing Technology

3.1 High Computational Overhead in Software-Based Encryption

Many existing systems implement cryptographic algorithms such as AES purely in software on general-purpose processors. Although this approach provides flexibility, it introduces significant computational overhead. AES encryption involves multiple complex operations such as SubBytes, ShiftRows, MixColumns, and AddRoundKey, which require several clock cycles when executed through software instructions. As a result, software-based encryption often leads to increased execution time and reduced system performance, particularly in resource-constrained embedded systems and IoT devices.

3.2 Limited Support for Cryptographic Acceleration in Conventional Processors

Traditional processor architectures were primarily designed for general-purpose computing rather than security applications. As a result, many processors lack built-in support for cryptographic operations. Without dedicated hardware acceleration, encryption and decryption tasks must be executed using multiple instructions, which increases processing time and power consumption. This limitation becomes critical in systems that require real-time secure data processing [3].

3.3 Higher Power Consumption in Security Implementations

In many embedded systems and IoT devices, power efficiency is an important design requirement. Software-based cryptographic implementations require multiple instruction executions and memory accesses, which increases energy consumption. This makes traditional encryption approaches less suitable for battery-powered devices where low power operation is essential.

3.4 Increased Latency in Secure Data Processing

When encryption algorithms are executed using standard processor instructions, multiple clock cycles are required to complete a single encryption round. This leads to higher latency, which can affect the performance of systems requiring high-speed secure communication such as wireless networks, financial systems, and secure embedded devices [1].

Hardware-based cryptographic accelerators integrated within processors can significantly reduce this latency by performing encryption operations in fewer clock cycles.

4. Methodology:

4.1 Design of the RISC-V Processor

The processor follows a single-cycle architecture, where every instruction completes in one clock cycle. The main functional blocks implemented in the design include:

Program Counter, Instruction Memory, Control Unit, Register File, Immediate Generator, ALU, AES Cryptographic Accelerator, Data Memory, Multiplexers and Control Signals. Each of these modules is coded as an independent Verilog module and connected through a data path.

Program Counter (PC)

The Program Counter stores the address of the current instruction being executed. Initially set to zero. After every clock cycle, it increments by 4 bytes to fetch the next instruction. PC provides the address to the instruction memory.

Instruction Memory

Instruction memory stores the RISC-V program instructions.

Operation:

The PC provides the address.
The instruction stored at that address is fetched

Control Unit

The Control Unit decodes the instruction opcode and generates control signals for other blocks. Inputs to control unit are opcode, funct7 and funct3. Outputs are control signals such as : RegWrite , MemRead, MemWrite, ALUSrc, MemtoReg, Branch, AES_Enable.

Register File

The register file stores temporary data used during instruction execution. The register file consists of 32 registers, each 32 bits wide, with two read ports and one write port. This values are sent to ALU or AES module

Immediate Generator

Some instructions use immediate values instead of registers.The immediate generator extracts the immediate bits from the instruction and converts them into 32-bit values.

Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations.

The ALU operation depends on:

opcode
funct3
Funct7

AES Cryptographic Accelerator

The AES module performs hardware-accelerated encryption operations. Instead of executing AES steps through many instructions, the processor executes a single custom instruction.

AES Functional Modules

The AES accelerator consists of the SubBytes, ShiftRows, MixColumns, and AddRoundKey modules. These modules are connected sequentially.

SubBytes Module

Performs byte substitution using the AES S-box lookup table. Input provided to SubBytes module is 8 bit data and output is substituted byte. This module is implemented using a case statement lookup table in Verilog.

ShiftRows Module

This module performs row shifting on the 4×4 AES state matrix.

Row 0 - shift 0

Row 1 - shift 1 byte

Row 2 - shift 2 byte

Row 3 - shift 3 byte

MixColumns Module

The MixColumns module performs a linear transformation on each column of the AES state matrix to provide diffusion. In this stage, each column of the 4×4 state matrix is treated as a polynomial and multiplied by a fixed matrix in the Galois Field GF(2^8). This operation mixes the bytes within each column, ensuring that changes in one byte affect multiple bytes in the output, thereby increasing the security of the encryption process.

AddRoundKey Module

AddRoundKey performs XOR operation between the state and the round key.

State = State XOR RoundKey

This step integrates the secret key into the encryption process.

Key Expansion (Key Schedule)

The Advanced Encryption Standard (AES) uses a key expansion algorithm to generate a set of round keys from the original secret key. Each round of the AES encryption process requires a unique round key to enhance the security of the encryption. The key expansion process ensures that the encryption key used in each round is different, thereby increasing resistance against cryptographic attacks.

For AES-128, the original key size is **128 bits**, and the encryption process consists of **10 rounds**. The key expansion algorithm generates **11 round keys**, where each round key is **128 bits** in length [1]. These round keys are derived from the initial key through a series of transformations involving substitution, rotation, and XOR operations.

The key expansion algorithm produces **44 words**, where each word consists of **32 bits** [1]. The first four words correspond to the original key, and the remaining words are generated iteratively using previously generated words.

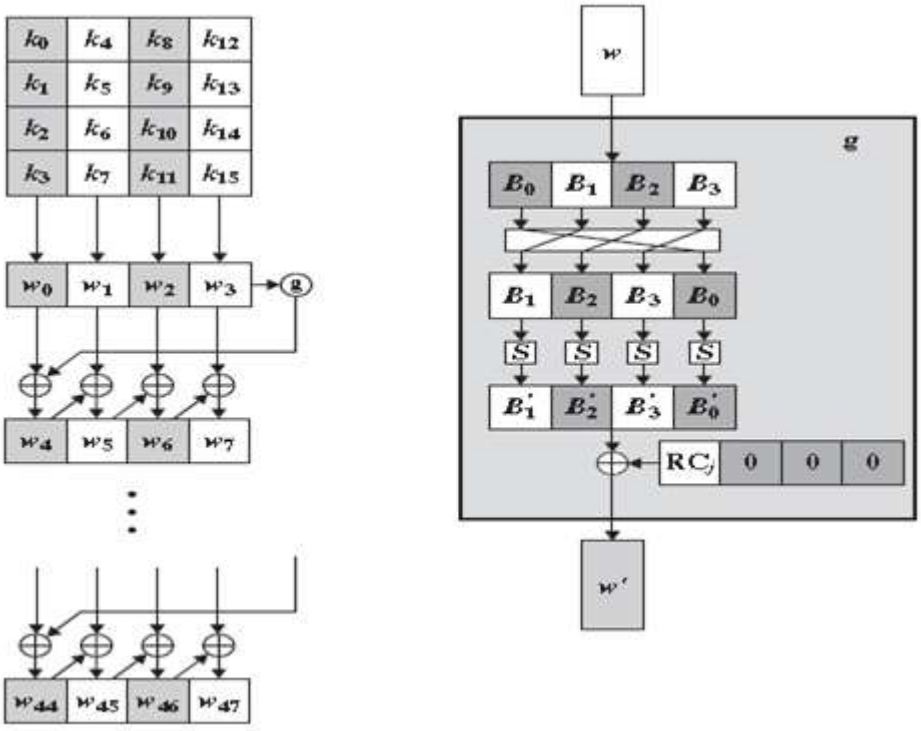


Figure 2 : AES Key Expansion [11]

Data Memory:

Data memory stores and retrieves data during program execution. Supported operations are lw and sw and control signals needed are MemRead and MemWrite.

4.2 Processor Execution Flow

The processor executes instructions using the following sequence:

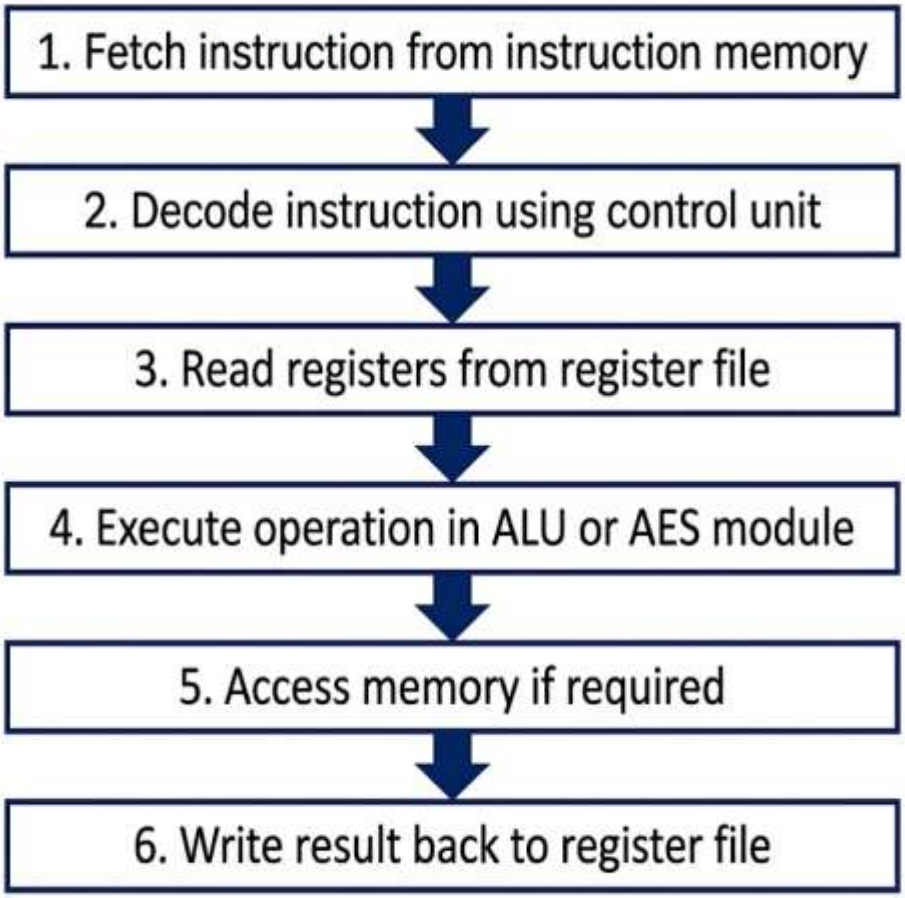


Figure 3: Execution Flow

This process completes in one clock cycle for each instruction.

4.3 Standard Instruction Set

Instr	Fmt	Opcode	funct3	funct7	Operation	Example	Notes
ADD	R	0110011	000	0000000	rd = rs1 + rs2	`add x3, x1, x2`	—
SUB	R	0110011	000	0100000	rd = rs1 − rs2	`sub x3, x2, x1`	funct7[5]=1 distinguishes from ADD
AND	R	0110011	111	0000000	rd = rs1 & rs2	`and x5, x3, x2`	Bitwise AND
OR	R	0110011	110	0000000	rd = rs1 | rs2	`or x6, x1, x2`	Bitwise OR
XOR	R	0110011	100	0000000	rd = rs1 ^ rs2	`xor x7, x3, x2`	Bitwise XOR
SLT	R	0110011	010	0000000	rd = (rs1 < rs2) ? 1 : 0	`slt x11, x1, x2`	Signed comparison
SLL	R	0110011	001	0000000	rd = rs1 << rs2[4:0]	`sll x8, x1, x2`	Logical shift left
SRL	R	0110011	101	0000000	rd = rs1 >> rs2[4:0]	`srl x9, x8, x1`	Logical shift right

SRA	R	0110011	101	0100000	rd = rs1 >>> rs2[4:0]	sra x10, x8, x1	Arithmetic shift right; sign-extended
ADDI	I	0010011	000	—	rd = rs1 + sign_ext(imm)	addi x1, x0, 10	12-bit signed immediate
ANDI	I	0010011	111	—	rd = rs1 & sign_ext(imm)	andi x15, x3, 15	—
ORI	I	0010011	110	—	rd = rs1 | sign_ext(imm)	ori x6, x0, 255	—
XORI	I	0010011	100	—	rd = rs1 ^ sign_ext(imm)	xori x7, x6, 240	—
SLTI	I	0010011	010	—	rd = (rs1 < imm) ? 1 : 0	slti x8, x1, 15	Signed comparison with immediate
SLLI	I	0010011	001	0000000	rd = rs1 << imm[4:0]	slli x13, x1, 3	Upper 7 bits of imm = 0000000
SRLI	I	0010011	101	0000000	rd = rs1 >> imm[4:0]	srli x14, x13, 2	Logical; upper 7 bits = 0000000
SRAI	I	0010011	101	0100000	rd = rs1 >>> imm[4:0]	srai x1, x1, 0	Arithmetic; upper 7 bits = 0100000
LW	I	0000011	010	—	rd = Mem[rs1 + imm]	lw x4, 0(x0)	Word load; addr must be 4-byte aligned

SW	S	0100011	010	–	Mem[rs1 + imm] = rs2	`sw x3, 0(x0)`	Word store; imm split across [31:25],[11:7]
BEQ	B	1100011	000	–	if rs1==rs2: PC = PC+4+imm	`beq x1, x2, +8`	Branch if equal
BNE	B	1100011	001	–	if rs1≠rs2: PC = PC+4+imm	`bne x1, x2, +8`	Branch if not equal
BLT	B	1100011	100	–	if rs1 < rs2 (signed): branch	`blt x1, x2, +8`	Signed less-than branch
BGE	B	1100011	101	–	if rs1 ≥ rs2 (signed): branch	`bge x2, x1, +8`	Signed greater-or-equal branch
JAL	J	1101111	–	–	rd = PC+4; PC = PC+imm	`jal x10, +8`	rd stores return address; 21-bit offset
LUI	U	0110111	–	–	rd = imm[31:12] << 12	`lui x9, 0x12345`	Loads upper 20 bits; lower 12 = 0

Table 1 : Standard Instruction Set Included in Processor

4.4 AES Custom Instruction Extension

Mnemonic	funct 3	Opcode	funct7	AES Step	RTL Operation
AESSUB	001	0001011	0000000	SubBytes on all 16 bytes of 128-bit data	`rd = SubBytes({rs1,rs2,rs1,rs2})[31:0]`
AESSHIFT	010	0001011	0000000	ShiftRows on 128-bit state	`rd = ShiftRows({rs1,rs2,rs1,rs2})[31:0]`
AESMIX	011	0001011	0000000	MixColumns on 128-bit state	`rd = MixColumns({rs1,rs2,rs1,rs2})[31:0]`
AESKEY	100	0001011	0000000	AddRoundKey: XOR data with key	`rd = (data ^ key)[31:0]`
AESENC	101	0001011	0000000	Full AES-128 encryption (10 rounds)	`rd = AES128_Encrypt(data, key)[31:0]`

Table 2: Custom AES Instructions

4.5 AES Data/Key Construction from Registers

Field	Value / Construction
128-bit data	`{ rs1[31:0], rs2[31:0], rs1[31:0], rs2[31:0] }`
128-bit key	`{ rs1[31:0], rs2[31:0], rs1[31:0], rs2[31:0] }`
Result written to rd	`aes_out[31:0]` **(lower 32 bits of 128-bit AES output)**
Key expansion	**11 round keys (RK0–RK10) generated combinationally from the 128-bit key via the AES-128 key schedule (RotWord + SubWord + Rcon XOR)**
Rcon values used	`01, 02, 04, 08, 10, 20, 40, 80, 1B, 36` **(rounds 1–10)**

Table 3 : Key Construction Logic

5. AES Encryption and Decryption block diagram

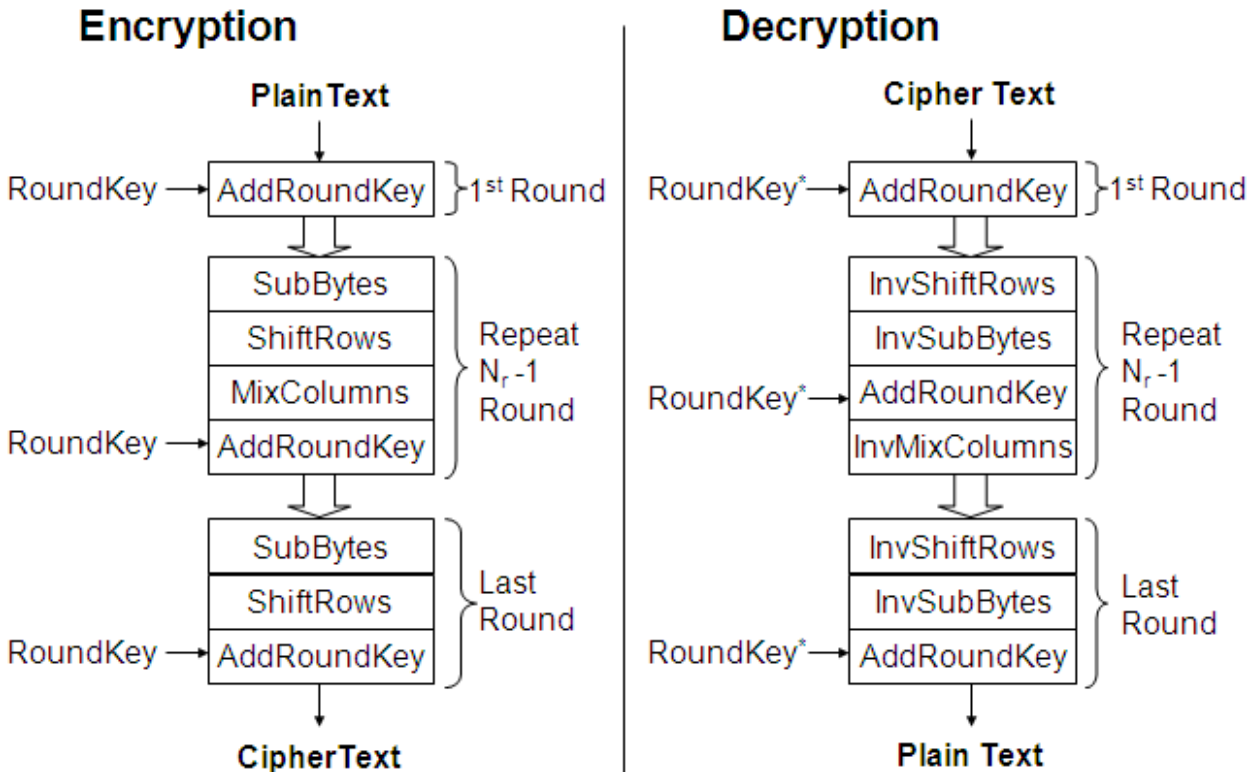


Figure 4 : AES-Encryption-Decryption-Flowchart [9]

6. Simulation Results and Implementation

6.1 Simulation of AES Block

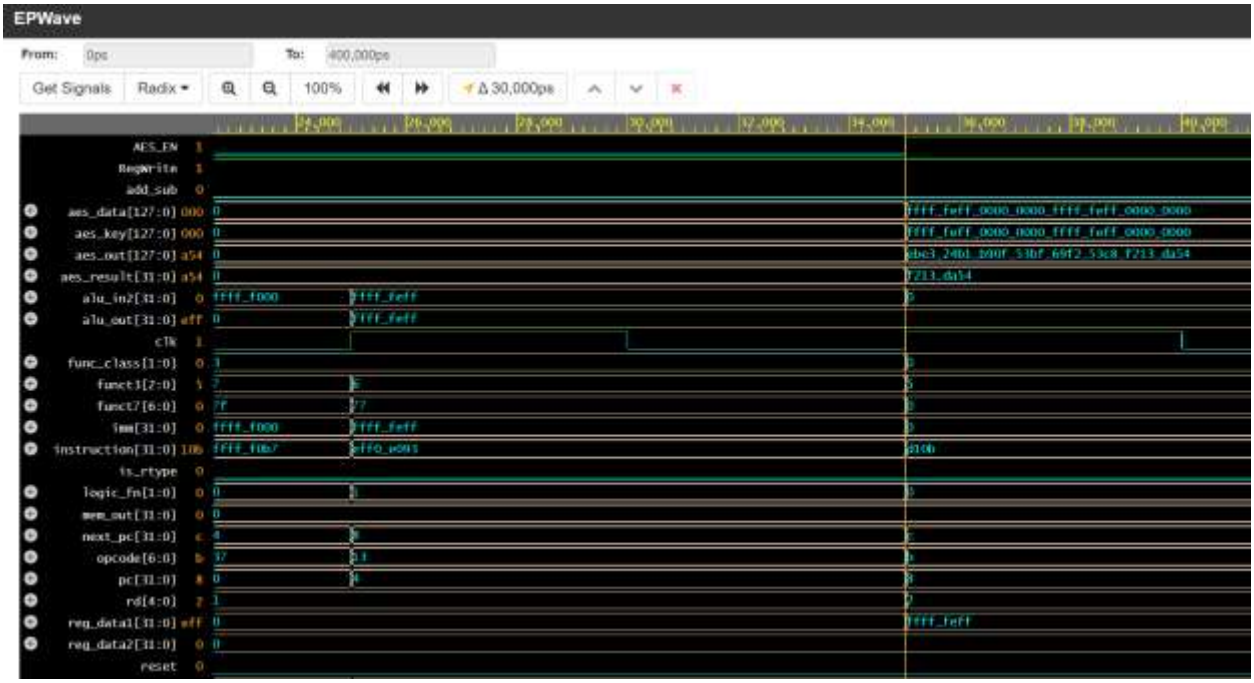


Figure 5 : Simulation results of AES Block

Instruction 1 : fffff0b7 - LUI x1,0xFFFFF

Instruction 2 : eff0e093 - ORI x1,x1,0xEFF

Hence after implementation of these two instructions the content of x1 becomes 0xFFFFFEFF

Instruction 3 : AES x2,x1,x0

rs1=x1= 0xFFFFFEFF

rs2=x0= 0X00000000

Hence , Aes_data = FFFFFEFF_00000000_FFFFFEFF_00000000_FFFFFEFF

Aes_key = FFFFFEFF_00000000_FFFFFEFF_00000000_FFFFFEFF

For which the encrypted data is :

Aes_out = EBE324B1_B90F53BF_69F253C8_F213DA54

The result displayed is true according to Advanced Encryption Standard

6.2 Simulation of Standard RV32I Instructions

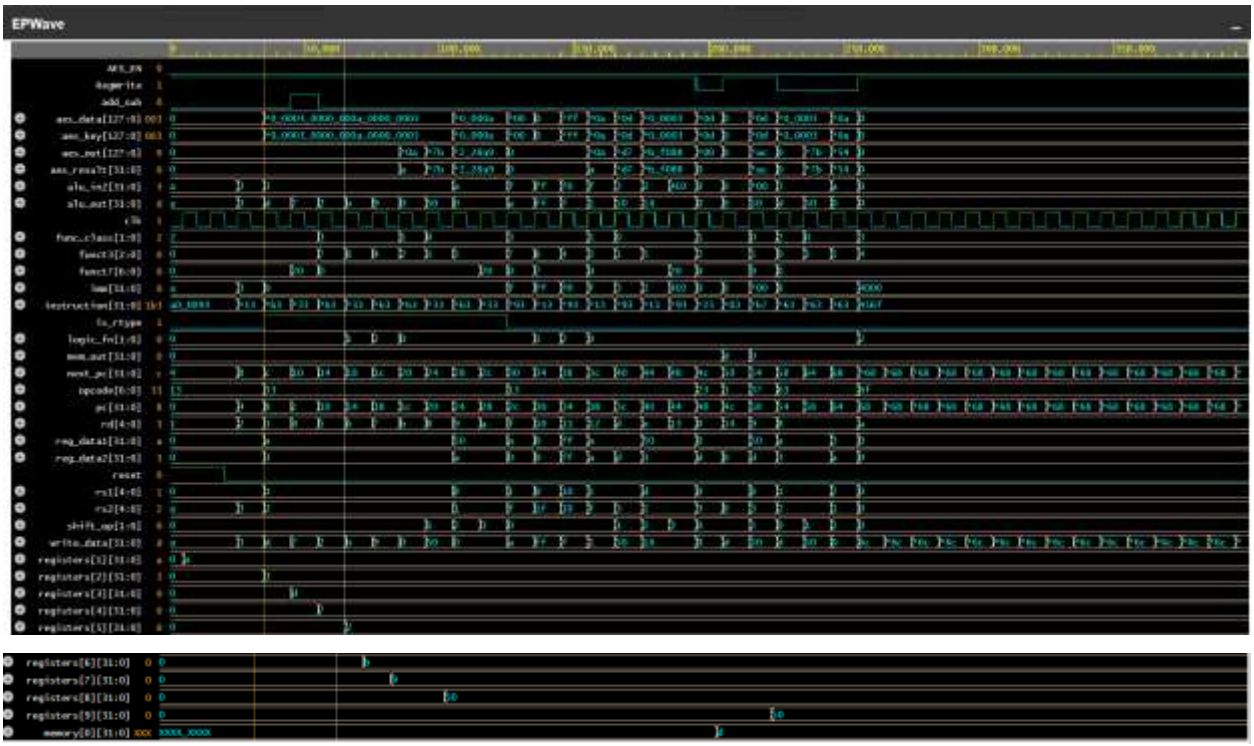


Figure 6 : Simulation result of RV32I Instructions

Instructions executed to verify the functionality

PC	Hex	Fmt	Assembly	Binary Encoding [fields separated by |]	Operation / Result	Notes
SETUP — Load values into registers						
0x000	00a00093	I	ADDI x1, x0, 10	000000001010 | 00000 | 000 | 00001 | 0010011	x1 = 0 + 10 = 10	Load decimal 10 into x1
0x004	00300113	I	ADDI x2, x0, 3	000000000011 | 00000 | 000 | 00010 | 0010011	x2 = 0 + 3 = 3	Load decimal 3 into x2

PC	Hex	Fmt	Assembly	Binary Encoding [fields separated by |]	Operation / Result	Notes
R-TYPE — Register Arithmetic						
0x008	002081b3	R	ADD x3, x1, x2	0000000 | 00010 | 00001 | 000 | 00011 | 0110011	x3 = 10 + 3 = 13	Add x1 and x2
0x00C	40208233	R	SUB x4, x1, x2	0100000 | 00010 | 00001 | 000 | 00100 | 0110011	x4 = 10 - 3 = 7	funct7[5]=1 distinguishes from ADD
0x010	0020f2b3	R	AND x5, x1, x2	0000000 | 00010 | 00001 | 111 | 00101 | 0110011	x5 = 1010 & 0011 = 0010 = 2	Bitwise AND
0x014	0020e333	R	OR x6, x1, x2	0000000 | 00010 | 00001 | 110 | 00110 | 0110011	x6 = 1010 | 0011 = 1011 = 11	Bitwise OR
0x018	0020c3b3	R	XOR x7, x1, x2	0000000 | 00010 | 00001 | 100 | 00111 | 0110011	x7 = 1010 ^ 0011 = 1001 = 9	Bitwise XOR
0x01C	0020a5b3	R	SLT x11, x1, x2	0000000 | 00010 | 00001 | 010 | 01011 | 0110011	x11 = (10<3)?1:0 = 0	Signed comparison
0x020	00209433	R	SLL x8, x1, x2	0000000 | 00010 | 00001 | 001 | 01000 | 0110011	x8 = 10 << 3 = 80	Shift left by x2[4:0]=3
0x024	001454b3	R	SRL x9, x8, x1	0000000 | 00001 | 01000 | 101 | 01001 |	x9 = 80 >> 10 = 0	Logical right shift

				0110011		
0x028	40145533	R	SRA x10, x8, x1	0100000 | 00001 | 01000 | 101 | 01010 | 0110011	x10 = 80 >>> 10 = 0	Arithmetic shift, sign extended

PC	Hex	Fmt	Assembly	Binary Encoding [fields separated by |]	Operation / Result	Notes
I-TYPE — Immediate Arithmetic						
0x02C	00f0f793	I	ANDI x15, x1, 15	000000001111 | 00001 | 111 | 01111 | 0010011	x15 = 10 & 15 = 10	Bitwise AND with immediate
0x030	0ff06813	I	ORI x16, x0, 255	000011111111 | 00000 | 110 | 10000 | 0010011	x16 = 0 | 255 = 255	Bitwise OR with immediate
0x034	0f084893	I	XORI x17, x16, 240	000011110000 | 10000 | 100 | 10001 | 0010011	x17 = 255 ^ 240 = 15	Bitwise XOR with immediate
0x038	00f0a913	I	SLTI x18, x1, 15	000000001111 | 00001 | 010 | 10010 | 0010011	x18 = (10<15)? 1:0 = 1	Signed less-than immediate
0x03C	00309693	I	SLLI x13, x1, 3	0000000 | 00011 | 00001 | 001 | 01101 | 0010011	x13 = 10 << 3 = 80	Shift amount in imm[4:0]
0x040	0026d713	I	SRLI x14, x13, 2	0000000 | 00010 | 01101 | 101 | 01110 | 0010011	x14 = 80 >> 2 = 20	Logical right shift immediate

0x044	4026d993	I	SRAI x19, x13, 2	0100000 | 00010 | 01101 | 101 | 10011 | 0010011	x19 = 80 >>> 2 = 20	funct7=0100000 distinguishes from SRLI

PC	Hex	Fmt	Assembly	Binary Encoding [fields separated by |]	Operation / Result	Notes
MEMORY — Store & Load						
0x048	00302023	S	SW x3, 0(x0)	0000000 | 00011 | 00000 | 010 | 00000 | 0100011	Mem[0+0] = x3 = 13	Store word to memory address 0
0x04C	00002a03	I	LW x20, 0(x0)	000000000000 | 00000 | 010 | 10100 | 0000011	x20 = Mem[0] = 13	Load word from memory address 0

PC	Hex	Fmt	Assembly	Binary Encoding [fields separated by |]	Operation / Result	Notes
U-TYPE — Upper Immediate						
0x050	123454b7	U	LUI x9, 0x12345	00010010001101000101 | 01001 | 0110111	x9 = 0x12345000	Upper 20 bits loaded; lower 12 = 0

PC	Hex	Fmt	Assembly	Binary Encoding [fields separated by |]	Operation / Result	Notes

BRANCH — Conditional Jumps						
0x054	00208463	B	BEQ x1, x2, +8	0000000 | 00010 | 00001 | 000 | 01000 | 1100011	**10==3? NO → no branch**	Branch NOT taken, PC=PC+4=88
0x058	00209463	B	BNE x1, x2, +8	0000000 | 00010 | 00001 | 001 | 01000 | 1100011	**10!=3? YES → PC=88+8=96**	Branch TAKEN, NOP at PC=92 skipped
0x05C	00000013	I	NOP (ADDI x0,x0,0)	000000000000 | 00000 | 000 | 00000 | 0010011	No operation	*SKIPPED by BNE above*
0x060	0020c463	B	BLT x1, x2, +8	0000000 | 00010 | 00001 | 100 | 01000 | 1100011	**10<3? NO → no branch**	Branch NOT taken, PC=PC+4=100
0x064	00115463	B	BGE x2, x1, +8	0000000 | 00001 | 00010 | 101 | 01000 | 1100011	**3>=10? NO → no branch**	Branch NOT taken, PC=PC+4=104

PC	Hex	Fmt	Assembly	Binary Encoding [fields separated by |]	Operation / Result	Notes
JUMP — Unconditional						
0x068	0000456f	J	JAL x10, +8	0 | 0000000100 | 0 | 00000000 | 01010 | 1101111	x10=108, PC=104+8=112	NOP at PC=108 skipped
0x06	0000001	I	NOP (ADDI	000000000000 | 00000 |	No operation	*SKIPPED by*

c	3		x0,x0,0)	000 | 00000 | 0010011		*JAL above*

Table 4 : Instructions executed to verify processor

8. CONCLUSION:

Conclusion

This project presented the design and implementation of a single-cycle RISC-V processor integrated with an AES-128 cryptographic accelerator using Verilog HDL. The processor architecture was developed by implementing essential components such as the program counter, instruction memory, register file, control unit, ALU, immediate generator, data memory, and branch logic. A subset of the RV32I instruction set including arithmetic, logical, memory access, and control instructions was successfully implemented and verified through simulation.

To enhance the processor's capability for secure applications, a dedicated AES hardware accelerator was integrated into the processor datapath. The AES module supports cryptographic operations including SubBytes, ShiftRows, MixColumns, AddRoundKey, and full AES encryption, along with a key expansion unit that generates round keys required for the AES-128 encryption process. The accelerator is activated through custom AES instructions, enabling efficient execution of encryption tasks directly in hardware.

Simulation results confirm that the processor correctly executes standard RISC-V instructions as well as AES cryptographic operations.

9. References

1. **National Institute of Standards and Technology (NIST)**, *Advanced Encryption Standard (AES)*, FIPS Publication 197, November 2001.
2. Joan Daemen and Vincent Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer, 2002.
3. David A. Patterson and Andrew Waterman, *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon LLC, 2017.
4. RISC-V International, *The RISC-V Instruction Set Manual Volume I: Unprivileged ISA Specification*, Version 20191213.
5. William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson Education, 7th Edition, 2017.
6. IEEE Xplore Digital Library, Research papers on *FPGA Implementation of AES Encryption and RISC-V Processors*, available at https://ieeexplore.ieee.org.
7. OpenCores, *AES Hardware Implementation Projects*, available at https://opencores.org.
8. Xilinx, *Vivado Design Suite User Guide*, for FPGA-based hardware design and simulation.
9. T. Prabhakar, B. Biswal, and V. Santhi, "Design and Analysis of Multimedia Communication System," *Conference Paper*, Dec. 2011.
10. Y. Liu, Z. Zhang, and H. Wang, "RISC-V Processor Core Design and Implementation," *ResearchGate*, 2022.
11. R. Kaur and H. Singh, "Implementation of AES Algorithm for Secure Data Communication," *International Journal of Computer Applications*, vol. 179, no. 27, pp. 20–24, 2018. Available: ResearchGate.